

SAP Fiori Elements

RAP Course, CAP Build

You Already Built RAP — Now Build CAP

SESSION 5 • UX406 • 2 HOURS

C_FIORD: Fiori Elements & FPM — 21–30% | Annotations — 11–20% | OData & BTP — 11–20%

UX406: Fiori Elements App Based on an OData V4 RAP Service (11 units, 5h 38min)

learning.sap.com/courses/getting-started-with-creating-an-sap-fiori-elements-app-based-on-an-odata-v4-rap-service



Why UX406 Reinforces Your Highest-Value Topics

UX406 hits the same three topics as UX405 — together they can account for up to ~60% of your C_FIORD score

Topic	Weight	Course	Session
SAP Fiori Elements & FPM	21–30%	UX405, UX406, UX410	TODAY ★
Fiori Elements Annotations	11–20%	UX405, UX406	TODAY ★
OData and SAP BTP	11–20%	UX405, UX406	TODAY ★

← UX406 feeds the SAME three exam topics as UX405, from the RAP side. Today: same annotations, CAP build.

Complete Unit Coverage — Today's Roadmap

All 11 units of UX406

U1

Fiori Elements Overview

Benefits, OData V4 advantages

31min

U8

Navigation Concept

Internal/external navigation — CAP syntax

15min

U2

ABAP RAP Overview

Already built live in Session 4 — RAP fundamentals

50min

U9

Configuring Object Pages

HeaderInfo, Facets, FieldGroup — CAP syntax

17min

U3

Configuring ABAP CDS Annotations

Backend annotation syntax — RAP side

1h 14min

U10

Flexible Programming Model

Custom sections, Building Blocks

37min

U4

SAP Fiori Tools Overview

App Generator, Page Map, Page Editor

42min

U11

Deploying an SAP Fiori Elements App

CAP deployment overview

26min

U5

Fiori Elements Floorplans

5 templates — quick review

16min

U6

General Settings: List Report/OP

FCL, draft handling, condensed mode

15min

U7

Configuring Tables

LineItem, criticality, actions — CAP syntax

15min

UX406

Fiori Elements Overview + RAP vs CAP Architecture

Units 1–2 • What is Fiori Elements, ABAP RAP fundamentals recap

U1: Fiori Elements Overview + U2: ABAP RAP vs CAP

Same core ideas as UX405 • RAP built in Session 4 • CAP built today

Fiori Elements Overview (U1) — Recap

Metadata-Driven UI	No hand-coded views — generated from OData V4 annotations
Five Floorplans	Same pre-built, tested, accessible templates as UX405
OData V4 Only	Not compatible with OData V2 — RAP or CAP, doesn't matter
Annotation-Driven	Developer writes annotations; Fiori Elements renders the UI
Backend-Agnostic	Works identically whether the OData V4 service is RAP or CAP

ABAP RAP vs CAP (U2) — Architecture

Data Model	RAP: Interface View (ABAP CDS) CAP: db/schema.cds
Business Logic	RAP: Behavior Def + class CAP: service.cds + .js
Service Exposure	RAP: Service Def + Binding CAP: service {} block
Runtime	RAP: ABAP system, on-prem/BTP CAP: Node.js/Java on BTP
Draft Handling	RAP: with draft; CAP: @odata.draft.enabled

⚠ Exam Trap ★

RAP and CAP are both valid, fully-supported ways to build the OData V4 service behind Fiori Elements. The exam tests whether you recognise which one a scenario describes, not that one is 'correct'. Cues: ABAP CDS, Eclipse ADT, Behavior Definition → RAP. Node.js, .cds files, cds watch, BTP → CAP.

UX406

Configuring Annotations + Tools + Templates

Units 3–5 • CAP CDS annotations, Fiori Tools, floorplan templates

U3: Configuring CAP CDS Annotations (Backend)

The CAP equivalent of the ABAP CDS annotation work from Session 4

Already Covered — RAP (Session 4)

- Interface View: define root view entity ZI_Flight_S4
- Consumption View: @UI.selectionField, @UI.linItem, @UI.facet
- Behavior Definition: managed implementation, actions, mapping
- Criticality: CASE WHEN written directly in the SELECT list
- Service Definition + Binding: Eclipse ADT, published via /IWFND/V4_ADMIN
- Full working example: your Session 4 Demonstration Evidence Log

Building Today — CAP CDS

- db/schema.cds: entity + calculated elements (same CASE WHEN idea)
- srv/flight-service.cds: service {} block, @odata.draft.enabled
- annotate ... with @(...): @UI.* terms, capitalized and plural
- actions {} on the entity = bound; service level = unbound
- No Eclipse ADT: cds watch auto-generates \$metadata + mock server
- Full working example: this slide's code block below

db/schema.cds — Calculated Element (CAP's CASE WHEN)

```
entity Flights : cuid, managed {  
  price : Decimal(9,2);  
  priceCriticality : Integer =  
    case when price <= 500 then 3  
    when price <= 1000 then 2 else 1 end;  
}
```

U4: SAP Fiori Tools + U5: Fiori Elements Floorplans

Identical tooling and templates regardless of backend — RAP or CAP

SAP Fiori Tools (U4) — in SAP BAS

Application Generator	Yeoman wizard → scaffolds full Fiori Elements project
Page Map	Visual editor — add/remove pages, set navigation routing
Page Editor	Click-to-configure UI: add columns, filter fields, sections
Guided Development	Step-by-step wizards for advanced features (micro charts etc.)
Service Modeler	Visualise & edit CDS model graphically

Fiori Elements Floorplans (U5)

List Report + Object Page	Most common — browse list, open detail. Any DB.
Overview Page (OVP)	Cards dashboard — KPIs from multiple data sources
Analytical List Page (ALP)	Chart + table + visual filter. SAP HANA only.
Worklist	Simplified list without complex filters. Any DB.
Form Entry Object Page	Single-entity form for create/edit use cases.

Exam Trap ★

Fiori Tools and the five floorplans don't care whether the OData V4 service behind them is RAP or CAP — same Application Generator, same Page Map, same templates. ALP still requires SAP HANA either way.

UX406

List Report Settings + Navigation

Units 6–8 • FCL, table columns, actions, internal/external navigation

U6: General Settings + U7: Configuring Tables

FCL, draft handling • @UI.LineItem, criticality, actions — CAP syntax

U6: General Settings (List Report/OP)

- Flexible Column Layout (FCL): List + Object Page side by side
- Draft handling: @odata.draft.enabled gives save-as-draft for free
- Condensed mode / inline creation: same UI5 table settings as RAP
- These settings live in the generated app, not the backend
- Auto-routing: Fiori Tools configures List Report → Object Page automatically
- manifest.json: routing config — same file structure, RAP or CAP
- Identical behaviour whether the OData V4 service is RAP or CAP

U7: Configuring Tables — CAP Syntax

- @UI.LineItem: table columns — same term as RAP, capitalized in CAP
- Criticality: { Value: price, Criticality: priceCriticality }
- Bound action: declared inside actions {} on the entity projection
- Unbound action: declared at service level, outside actions {}
 - Same 1=red 2=amber 3=green 0=neutral scale as RAP
- @UI.DataFieldForAction: adds the toolbar/inline button — same term as RAP
- Same bound vs unbound logic as RAP's Behavior Definition, different syntax

⚠ Exam Trap ★

Bound vs unbound is a syntax-position rule in CAP: declared inside actions{} on the entity → bound (needs a selected record). Declared at service level, outside actions{} → unbound (independent). Same logic as RAP's Behavior Definition, just expressed differently.

U8: Navigation Concept

Internal, external, and intent-based navigation — CAP syntax

Internal Navigation (U8)

- List Report → Object Page: automatic, zero annotation work
- Configured entirely by Fiori Tools when the app is generated
- Navigation to a related entity: expose a to-many association
- Inside a Facet as a sub-table — no special nav annotation needed
- Breadcrumb: Fiori Elements auto-renders back-navigation in the header
- Same mechanism as the RAP build — this is a shared FE concept
- No manifest.json routing edits needed for standard navigation

External / Intent-Based Navigation (U8)

- External navigation: leaves the current app for a separate FLP tile/app
- Intent-based: semantic object + action (e.g. Flight-displayFactSheet)
- @UI.DataFieldWithIntentBasedNavigation: same term as RAP
- SemanticObject + Action: the two required annotation properties
- Resolution depends on the Fiori Launchpad's target mapping/catalog
- Testable only in a real Launchpad — not the raw OData preview
- Critical for exam: the annotation term is identical, RAP or CAP

`@UI.DataFieldWithIntentBasedNavigation — CAP`

```
annotate FlightService.Flights with @(
  UI.LineItem: [{ $Type: 'UI.DataFieldWithIntentBasedNavigation',
    SemanticObject: 'Flight', Action: 'displayFactSheet' }]
);
```

UX406

Object Pages + FPM + Deployment

Units 9–11 • HeaderInfo, Facets, Building Blocks, deploying to BTP

U9: Configuring Object Pages

@UI.HeaderInfo • @UI.Facets • @UI.FieldGroup — same terms as RAP

Object Page Header (U9)

- @UI.HeaderInfo.TypeName / TypeNamePlural: singular/plural labels
- @UI.HeaderInfo.Title: property shown as the main title
- @UI.HeaderInfo.Description: sub-title shown under the title
- Same four sub-properties as RAP — identical annotation vocabulary
- Header renders automatically once @UI.HeaderInfo is set
- Optional: @UI.HeaderFacets embeds KPIs/micro charts in the header
- Criticality still works the same way here as on List Report columns

Object Page Body (U9)

- @UI.Facets: top-level body structure — defines sections/tabs
- \$Type: 'UI.ReferenceFacet': points at a FieldGroup, Table, or Chart
- \$Type: 'UI.CollectionFacet': groups facets into one section
- @UI.FieldGroup: groups fields together inside a form section
- Target: '@UI.FieldGroup#Qualifier' links the facet to its fields
- Structurally identical to RAP's @UI.facet — different property names
- Full example built live on the Flight scenario — see the code below

@UI.HeaderInfo + @UI.Facets — CAP Example

```
annotate FlightService.Flights with @(
  UI.HeaderInfo: { TypeName: 'Flight',
    Title: { Value: connectionID } },
  UI.Facets: [{ $Type: 'UI.ReferenceFacet',
    Target: '@UI.FieldGroup#GeneralData' }]);
```

U10: Flexible Programming Model + U11: Deployment

FPM is backend-agnostic • deployment genuinely differs

U10: Flexible Programming Model

- FPM = way to add CUSTOM UI elements to a Fiori Elements app
- Extension points: Custom Section, Column, Action, Header Facet
- Custom code runs alongside generated FE content — not a replacement
- Building Blocks: macros:Table, macros:Chart, macros:FilterBar, macros:Field
- Same xmlns:macros='sap.fe.macros' namespace — RAP or CAP, no difference
- Lives in webapp/ext + manifest.json — the UI project, not the backend
- Key rule: prefer annotations first; use FPM only for what they can't do

U11: Deploying an SAP Fiori Elements App

- CAP deployment is a different path from RAP's transport promotion
- cds build: compiles the CDS model, packages the service
- cds add mta: scaffolds a Multi-Target Application (mta.yaml)
- mbt build then cf deploy: builds and deploys the MTA to Cloud Foundry
- RAP equivalent: transport + /IWFND/MAINT_SERVICE or V4_ADMIN publish
- Both paths ultimately expose the same OData V4 service on SAP BTP
- Full deployment depth for both: Session 6 (UX410, ≤10% weight)

⚠ Exam Trap ★

FPM is the one topic where the answer is genuinely 'doesn't matter which backend' — don't look for a RAP/CAP angle that isn't there. Deployment mechanics DO differ and ARE UX410 exam territory — know that they differ, but the deep how-to isn't this session's job.

Know These Cold — UX406 (RAP vs CAP)

U1–U3 — Overview + RAP vs CAP + Annotations

- Data model: RAP Interface View | CAP db/schema.cds
- Service exposure: RAP Service Def+Binding | CAP service {} block
- Filter bar: RAP @UI.selectionField | CAP @UI.SelectionFields
- Table columns: RAP @UI.lineltem | CAP @UI.Lineltem
- 5 templates, same set either way: LR+OP, OVP, ALP, Worklist, Form Entry
- ALP requires SAP HANA — true for List Report on Any DB, either backend

U4–U8 — Settings + Tables + Navigation

- Bound action: RAP in Behavior Definition | CAP actions{} on entity
- Unbound action: RAP static action | CAP action at service level
- Criticality: 1=red 2=amber 3=green 0=neutral — same scale, both
- @UI.DataFieldForAction → custom action button, same term either way
- Draft: RAP with draft; | CAP @odata.draft.enabled
- @UI.DataFieldWithIntentBasedNavigation — identical term, both backends

U9–U11 — Object Pages + FPM + Deployment

- @UI.HeaderInfo → title/description, same term either backend
- @UI.Facets → body structure, same term either backend
- @UI.FieldGroup → form fields, same term either backend
- FPM: Custom Section/Column/Action — 100% identical, backend-agnostic
- macros:Table / Chart / FilterBar — same namespace, same syntax
- Deployment: RAP = transport/V4_ADMIN | CAP = cds build + mta + cf

Before Session 6 — Action Items

Session 6 covers UX410: Deployment & Testing

Next Session: Session 6 – Deployment & Testing (UX410) | 2 Hours

1

Complete UX406 on SAP Learning

All 11 units (5h 38min) — see UX406 on SAP Learning (full link on the Resources slide)

2

Build the CAP counterpart in SAP BAS

cds init a new project, add the Flight schema from this guide, generate the app with the Application Generator

3

Compare RAP vs CAP side by side

For every RAP annotation you know cold from Session 4, write its CAP equivalent from memory

4

Practice recognising RAP vs CAP from a scenario

ABAP CDS + Eclipse ADT = RAP. Node.js + .cds files + cds watch = CAP. Drill until automatic.

5

Preview Session 6 (UX410) — Deployment

CAP: cds build + mta.yaml + cf deploy. RAP: transport requests + /IWFND/MAINT_SERVICE.

Questions?

Session 5 • UX406 • SAP Fiori Elements Based on OData V4 (RAP Course, CAP Build)

UX406 on SAP Learning —	learning.sap.com/courses/getting-started-with-creating-an-sap-fiori-elements-app-based-on-an-odata-v4-rap-service
Fiori Elements V4 Reference —	sapui5.hana.ondemand.com/sdk/#/topic/7619517a924f4e859801e3a5d6a5d9f5
CAP Documentation —	cap.cloud.sap/docs
SAP Fiori Tools Docs —	help.sap.com/docs/sap-fiori-tools
SAP BTP Trial (BAS) —	account.hanatrial.ondemand.com
SAP Community Fiori —	community.sap.com/topics/fiori
Fiori Apps Library —	fioriappslibrary.hana.ondemand.com

Next: Session 6 – Deployment & Testing (UX410)